UPDATE and DELETE Operations

We will use one common collection throughout the practical:

```
college
  └── students
```

The students will learn:

1. Insert sample records
2. Display records
3. `updateOne()`
4. `updateMany()`
5. `$set`
6. `$inc`
7. `$unset`
8. `$rename`
9. `$push`
10. `$pull`
11. `replaceOne()`
12. `deleteOne()`
13. `deleteMany()`
14. Delete all documents
15. Difference between update and delete methods
16. Practical exercises

PART 1 — Start MongoDB Shell

First make sure the MongoDB server is running.

Open CMD:

```
mongosh
```

You should get:

```
test>
```

PART 2 — Select Database

Type:

```
use college
```

Output:

```
switched to db college
```

Check:

```
db
```

Output:

```
college
```

PART 3 — Create Sample Student Data

For today's practical, let's insert several students.

```
db.students.insertMany([
    {
        rollno: 101,
        name: "Rahul",
        age: 20,
        course: "BCA",
        semester: 5,
        city: "Anjar",
        marks: 75,
        fees: 45000
    },
    {
        rollno: 102,
        name: "Priya",
        age: 19,
        course: "BCA",
        semester: 3,
        city: "Gandhidham",
        marks: 82,
        fees: 45000
    },
    {
        rollno: 103,
        name: "Amit",
        age: 21,
        course: "BCA",
        semester: 5,
        city: "Bhuj",
        marks: 68,
        fees: 45000
    },
    {
        rollno: 104,
        name: "Neha",
        age: 20,
        course: "BCA",
        semester: 3,
        city: "Anjar",
        marks: 91,
        fees: 45000
    },
    {
```

```
        rollno: 105,
        name: "Karan",
        age: 22,
        course: "BCA",
        semester: 6,
        city: "Mandvi",
        marks: 59,
        fees: 45000
    }
])
```

Check:

```
db.students.find()
```

For easier reading:

```
db.students.find().pretty()
```

PART 4 — UPDATE OPERATIONS

MongoDB provides several methods for updating documents.

The most important are:

```
updateOne()
updateMany()
replaceOne()
```

We also use update operators such as:

```
$set
$inc
$unset
$rename
$push
$pull
```

---

1. `updateOne()`

Meaning

`updateOne()` updates **only one document** that matches the condition.

Syntax

```
db.collection.updateOne(
    { condition },
    { update operation }
)
```

Example 1 — Change student's city

Currently Rahul has:

```
city: "Anjar"
```

Let's change it to Gandhinagar.

```
db.students.updateOne(
    { rollno: 101 },
    { $set: { city: "Gandhinagar" } }
)
```

Output will be similar to:

```
{
    acknowledged: true,
    matchedCount: 1,
    modifiedCount: 1
}
```

Check:

```
db.students.findOne({ rollno: 101 })
```

You should see:

```
{
    rollno: 101,
    name: 'Rahul',
    ...
    city: 'Gandhinagar'
}
```

Explain to students

Here:

```
{ rollno: 101 }
```

is the **condition**.

And:

```
{ $set: { city: "Gandhinagar" } }
```

is the **update operation**.

---

2. `$set` Operator

`$set` is one of the most frequently used MongoDB update operators.

It is used to **add a new field or change an existing field**.

Syntax

```
{ $set: { field: value } }
```

Example

Change Rahul's marks:

```
db.students.updateOne(
    { rollno: 101 },
    { $set: { marks: 85 } }
)
```

Check:

```
db.students.findOne({ rollno: 101 })
```

Now:

```
marks: 85
```

---

3. Updating Multiple Fields

We can update several fields at once.

```
db.students.updateOne(
    { rollno: 101 },
    {
        $set: {
            city: "Bhuj",
            marks: 88,
            semester: 6
        }
    }
)
```

Check:

```
db.students.findOne({ rollno: 101 })
```

Three fields have been changed.

---

4. Add a New Field Using `$set`

Suppose students currently don't have a `mobile` field.

Add it:

```
db.students.updateOne(
    { rollno: 101 },
    { $set: { mobile: "9876543210" } }
)
```

MongoDB creates the field automatically.

This is an important concept for students:

MongoDB is schema-flexible. We can add a new field to a document without altering a table structure as we would in a traditional relational database.

---

5. `$inc` — Increment a Value

`$inc` means **increment**.

Suppose Rahul's marks are:

```
88
```

Increase by 5:

```
db.students.updateOne(
    { rollno: 101 },
    { $inc: { marks: 5 } }
)
```

Now marks become:

```
93
```

Another example

Increase fees by ₹2,000:

```
db.students.updateOne(
    { rollno: 101 },
    { $inc: { fees: 2000 } }
)
```

If fees were:

`45000`

they become:

`47000`

Important

`$inc` does not mean "set value to".

```
$inc: { marks: 5 }
```

means:

```
old marks + 5
```

---

6. `$unset` — Remove a Field

Suppose Rahul has:

```
mobile: "9876543210"
```

We want to remove the `mobile` field.

```
db.students.updateOne(
    { rollno: 101 },
    { $unset: { mobile: "" } }
)
```

Check:

```
db.students.findOne({ rollno: 101 })
```

The `mobile` field will disappear.

Important teaching point

`$unset` removes the **field**, not the entire document.

---

7. `$rename` — Rename a Field

Suppose we want to rename:

```
city
```

to:

```
address
```

Run:

```
db.students.updateOne(
    { rollno: 101 },
    { $rename: { "city": "address" } }
)
```

Check:

```
db.students.findOne({ rollno: 101 })
```

Now:

```
address: "Bhuj"
```

instead of:

```
city: "Bhuj"
```

---

8. `updateMany()`

This is very important.

`updateMany()` updates **all documents matching the condition**.

Example

Suppose we want to change the fees of all BCA students:

```
db.students.updateMany(
    { course: "BCA" },
    { $set: { fees: 50000 } }
)
```

Output might be:

```
{
    acknowledged: true,
    matchedCount: 5,
    modifiedCount: 5
}
```

Check:

```
db.students.find({}, { name: 1, fees: 1 })
```

All matching students now have:

```
fees: 50000
```

---

9. Update Students of a Particular Semester

For example, increase marks of all Semester 5 students by 5:

```
db.students.updateMany(
    { semester: 5 },
    { $inc: { marks: 5 } }
)
```

This is a very good example to demonstrate the difference between:

`updateOne()`

and:

`updateMany()`

---

10. Update Using Multiple Conditions

We can use multiple conditions.

For example:

Increase marks by 10 for BCA students whose marks are less than 70.

```
db.students.updateMany(
    {
        course: "BCA",
        marks: { $lt: 70 }
    },
    {
        $inc: { marks: 10 }
    }
)
```

Here:

`$lt`

means:

`less than`

---

11. Update with `$gte`

Increase marks by 2 for students having marks greater than or equal to 80:

```
db.students.updateMany(
    { marks: { $gte: 80 } },
    { $inc: { marks: 2 } }
)
```

`$gte` means:

Greater Than or Equal To

---

12. `replaceOne()`

`replaceOne()` is different from `$set`.

It **replaces the entire document** that matches the condition.

Example:

```
db.students.replaceOne(
    { rollno: 105 },
    {
        rollno: 105,
        name: "Karan",
        age: 22,
        course: "BCA",
        semester: 6,
        city: "Bhuj",
        marks: 75,
        fees: 50000
    }
)
```

Important teaching point

With `replaceOne()`, the old document is replaced by the new document.

If the original document had extra fields such as:

```
mobile
email
address
```

and they are not included in the replacement document, they will be removed.

So:

`$set`

➡️ changes selected fields.

`replaceOne()`

➡️ replaces the complete document.